

DevPulse — *Real-time CI/CD Observability & Alerting Platform*

Version: **1.0.0**

Status: **Draft**

Author: **Engineering**

Date: **May 2026**

◆ Overview

DevPulse is a self-hosted platform that gives engineering teams real-time visibility into their CI/CD pipelines, with intelligent alerting, live log streaming, and multi-channel notifications — built to demonstrate production-grade system design patterns.

OBJECTIVES

- 01 Ingest build events via webhooks and polling from GitHub/GitLab CI
- 02 Process events asynchronously through a task queue (Celery + Redis)
- 03 Stream live build updates to dashboards via WebSocket pub/sub
- 04 Fan out smart notifications across Email, Slack, and in-app channels
- 05 Provide flaky test detection and MTTR analytics per service

USER PERSONAS

Platform Engineer

Monitor all pipelines across teams from one view

Developer

Get instant alerts when their branch breaks

Engineering Manager

Track reliability trends and MTTR over time

◆ Functional Requirements

FR-01 Webhook & Event Ingestion

- | | | |
|---------|----|--|
| FR-01.1 | P0 | Expose a POST /webhooks/{provider} endpoint accepting GitHub and GitLab CI event payloads. |
| FR-01.2 | P0 | Validate incoming webhook HMAC-SHA256 signatures; reject invalid requests with 401. |
| FR-01.3 | P0 | Deduplicate webhook deliveries using the provider's delivery ID stored in Postgres. |
| FR-01.4 | P1 | Return 202 Accepted immediately and enqueue processing as an async Celery task. |
| FR-01.5 | P1 | Store raw payload in Postgres with parsed fields (status, branch, commit SHA, duration). |

FR-02 CI API Polling

- | | | |
|---------|----|--|
| FR-02.1 | P1 | Poll CI provider APIs every 5 seconds for active builds, backing off to 60s when idle. |
| FR-02.2 | P1 | Implement exponential backoff (max 5 min) on 4xx/5xx API errors with jitter. |
| FR-02.3 | P2 | Allow per-project polling interval configuration via the settings UI. |
| FR-02.4 | P2 | Log polling failures to a dead-letter store for operator review. |

FR-03 Async Task Queue

- | | | |
|---------|----|---|
| FR-03.1 | P0 | Process all webhook events through Celery workers backed by Redis broker. |
| FR-03.2 | P0 | Implement at-least-once delivery with idempotency guards on task execution. |
| FR-03.3 | P1 | Provide a dead-letter queue (DLQ) for tasks that fail after 3 retries. |
| FR-03.4 | P1 | Fan out notification tasks per channel (email, Slack, in-app) as independent sub-tasks. |
| FR-03.5 | P2 | Celery Beat schedules daily/weekly digest emails per user preference. |

FR-04 Pub/Sub & Live Streaming

- | | | |
|---------|----|---|
| FR-04.1 | P0 | Publish build state changes to a Redis channel keyed by project ID. |
| FR-04.2 | P0 | Django Channels consumer subscribes and pushes updates over WebSocket to connected clients. |
| FR-04.3 | P1 | Stream build log lines in real-time to the dashboard log viewer. |
| FR-04.4 | P1 | Fallback to Server-Sent Events (SSE) for clients that cannot maintain WebSocket. |
| FR-04.5 | P2 | Support room-based isolation so users only receive events for their org's projects. |

FR-05 Notification Engine

- | | | |
|---------|----|---|
| FR-05.1 | P0 | Users define alert rules: conditions (branch, status, consecutive failures) + channels. |
| FR-05.2 | P0 | Evaluate alert rules inside Celery tasks after each build event is processed. |
| FR-05.3 | P0 | Deliver notifications to Email (SMTP/SES), Slack (outbound webhook), and in-app bell. |
| FR-05.4 | P1 | Respect per-user notification quiet hours; queue messages for delivery after window. |
| FR-05.5 | P1 | Track notification delivery status (sent, failed, acknowledged) in Postgres. |
| FR-05.6 | P2 | Support Web Push notifications for PWA-installed dashboard. |
| FR-05.7 | P2 | Scheduled digest: summarise last 24h of failures per project, sent via Celery Beat. |

FR-06 Dashboard & Analytics

- | | | |
|---------|----|--|
| FR-06.1 | P0 | Live build timeline showing status, branch, duration and commit for last 50 runs. |
| FR-06.2 | P1 | Flaky test leaderboard: rank tests by pass/fail variance over a rolling 30-day window. |
| FR-06.3 | P1 | MTTR chart (mean time to recovery) per service, filterable by date range. |
| FR-06.4 | P1 | Notification history log with delivery status and rule that triggered each alert. |
| FR-06.5 | P2 | Export build data as CSV for a selected date range. |

FR-07 Auth & Multi-Tenancy

FR-07.1	P0	JWT-based authentication with refresh token rotation.
FR-07.2	P0	Projects belong to organisations; users can belong to multiple orgs with RBAC (Admin/Member/Viewer).
FR-07.3	P1	API keys per project for webhook authentication without user credentials.
FR-07.4	P2	OAuth 2.0 social login via GitHub.

P0

= Must have

P1

= Should have

P2

= Nice to have

Non-Functional Requirements

⚡ Performance

- ▶ Webhook endpoint must respond with 202 in < 100ms (p99) regardless of queue depth.
- ▶ Live dashboard updates must reach the browser within 500ms of a build state change.
- ▶ REST API responses (list endpoints) must complete in < 300ms (p95) under 200 concurrent users.
- ▶ Celery tasks must process a single webhook event end-to-end (parse → DB write → notify enqueue) in < 2s.

🔄 Scalability

- ▶ Worker pool must scale horizontally; adding workers should linearly increase queue throughput.
- ▶ Redis pub/sub layer must support at least 500 concurrent WebSocket connections per instance.
- ▶ Postgres schema uses indexes on (project_id, created_at) and (status, branch) for query performance at 10M+ rows.
- ▶ Polling scheduler distributes jobs across workers using consistent hashing to avoid thundering herd.

● Reliability

- ▶ At-least-once delivery guarantee for all Celery tasks with idempotency enforced at the DB level.
- ▶ DLQ retains failed tasks for 7 days with full payload for replay.
- ▶ Django Channels reconnects automatically on WebSocket drop; clients receive missed events on reconnect.
- ▶ Webhook deduplication window of 24h prevents duplicate processing of replayed deliveries.
- ▶ System must tolerate Redis unavailability gracefully: API remains functional, queue tasks accumulate.

⬡ Security

- ▶ All webhook signatures verified using constant-time HMAC comparison (no timing attacks).
- ▶ JWT access tokens expire in 15 minutes; refresh tokens in 7 days with rotation on use.
- ▶ All inter-service communication uses TLS; secrets stored in environment variables, never in code.
- ▶ RBAC enforced at the Django view layer; row-level checks prevent cross-org data access.
- ▶ Rate limit webhook endpoint to 1,000 req/min per project API key using Redis token bucket.
- ▶ Audit log for all RBAC-sensitive actions (rule create/delete, member add/remove) stored in Postgres.

◆ Observability

- ▶ Structured JSON logging (timestamp, request_id, user_id, duration) for all Django views and Celery tasks.
- ▶ Celery task metrics (enqueue rate, processing time, failure rate) exposed as Prometheus metrics.
- ▶ Health check endpoints: /health/live and /health/ready for container orchestration.
- ▶ Distributed tracing via OpenTelemetry spans across webhook → queue → worker → notification chain.

◇ Developer Experience

- ▶ docker-compose.yml spins up full stack (Django, Celery, Beat, Channels, Redis, Postgres) in one command.
- ▶ Seed script generates 30 days of synthetic build history for local dashboard development.
- ▶ Webhook event simulator CLI tool replays captured GitHub/GitLab payloads against local server.
- ▶ OpenAPI 3.0 spec auto-generated from Django REST Framework views; served at /api/docs.

▣ Tech Stack

BACKEND API	Django 5 + Django REST Framework	Batteries-included, mature ORM, excellent ecosystem for auth, serialization, and schema generation.
ASYNC WORKERS	Celery 5 + Redis Broker	Industry-standard task queue with Beat scheduler, retry logic, and Prometheus integration.
REAL-TIME LAYER	Django Channels + Redis pub/sub	

		Extends Django with ASGI, WebSocket support, and channel layers backed by Redis.
DATABASE	PostgreSQL 16	ACID guarantees for build records, JSONB for raw payloads, native window functions for analytics.
CACHE / BROKER	Redis 7	Serves triple duty: Celery broker, Django Channels layer, and API response cache.
FRONTEND	React 18 + TanStack Query + Recharts	TanStack Query handles REST polling + cache invalidation; native WebSocket hook for live updates.
AUTH	djangorestframework-simplejwt	Lightweight JWT with refresh rotation; pairs cleanly with DRF permission classes.
CONTAINERISATION	Docker + docker-compose	Reproducible dev environment; each service (api, worker, beat, channels) in its own container.

▲ Milestones

Phase 1 · Week 1-2

Foundation

- ✓ Django project scaffold with Postgres, JWT auth, RBAC models
- ✓ Webhook receiver endpoint with HMAC validation and deduplication
- ✓ Celery + Redis task queue wired up; basic worker processes events
- ✓ docker-compose for full local stack

Phase 2 · Week 3-4

Real-time Core

- ✓ Django Channels ASGI server with Redis channel layer
- ✓ Pub/sub: worker publishes build events → Channels → WebSocket
- ✓ React dashboard skeleton with live build timeline via WebSocket
- ✓ CI API polling with exponential backoff via Celery Beat

Phase 3 · Week 5-6

Notifications

- ✓ Alert rule engine: condition evaluator in Celery tasks
- ✓ Email delivery via SMTP with retry logic and DLQ
- ✓ Slack outbound webhook integration
- ✓ In-app notification bell with unread count via WebSocket
- ✓ Quiet hours and per-channel preference store

Phase 4 · Week 7-8

Analytics & Polish

- ✓ Flaky test detection algorithm and leaderboard UI
- ✓ MTTR chart with date-range filter (Recharts)
- ✓ Prometheus metrics endpoint + structured logging
- ✓ OpenAPI docs, seed script, webhook simulator CLI
- ✓ README with architecture diagram and demo GIF

⚠ Constraints & Risks

RISK	IMPACT	MITIGATION
Redis single point of failure	High	Add Redis Sentinel or use a managed Redis (ElastiCache) in production; design queue to tolerate downtime gracefully.
Webhook flood / abuse	Medium	Rate limit per API key (Redis token bucket), validate signatures before any DB write, reject oversized payloads early.
CI provider API rate limits	Medium	Implement per-provider rate limit tracking in Redis; back off proactively before hitting 429s.
WebSocket connection storms on deploy	Medium	

		Stagger reconnect with jitter on the client; Channels layer absorbs state, clients re-subscribe on connect.
RISK Notification delivery failures	IMPACT Low	MITIGATION DLQ captures all failed notification tasks; operator can replay. Track delivery status in DB for user visibility.